

Assignment 3b

Title: API for CRUD operations

Problem Statement: Create four API using Node.JS, ExpressJS and MongoDB for CURD Operations for Register User, Login User, Show User Data.

Objective: Use node JS, express, mongo DB and mongoose libraries to create CRUD operations for user administration.

S/W Packages and H/W apparatus used:

OS: Ubuntu/Windows, node JS, Express, Mongo DB, Postman, VSCode editor.

PC with the configuration as Pentium IV 1.7 GHz. 128M.B RAM, 40 G.B HDD, 15’’Color Monitor, Keyboard, Mouse.

References:

1. Adam Bretz & Colin J Ihrig, Full Stack Javascript Development with MEAN, SPD, First Edition, ISBN:978-0992461256.

Theory:

CRUD is an acronym for Create, Read, Update and Delete. It is a set of operations we get servers to execute (POST, GET, PUT and requests DELETE respectively). This is what each operation does:

- Create (POST)- **Make something**
- Read (GET)- **Get something**
- Update (PUT)- **Change something**
- Delete (DELETE)- **Remove something**

Step 1: Creating the Application

- Create a new folder for the application. *mkdir crud-node-express*

- Initialize the application with a package.json file

Go to the root folder of your application and type `npm init` to initialize your app with a package.json file. `cd crud-node-express npm init`

Step 2: Install dependencies

Install express, mongoose, and body-parser modules. `npm install express body-parser mongoose --save`

Setting up the Web Server

Create a new file named `server.js` in the root folder of the application with the following contents:

```
const express = require('express'); const bodyParser = require('body-parser'); const
app = express(); app.use(bodyParser.urlencoded({ extended: true }))
app.use(bodyParser.json()) app.get('/', (req, res) => { res.json({ "message": "Hello
Crud Node Express" }); }); app.listen(3000, () => { console.log("Server is listening on
port 3000"); });
```

Step 3: Configuring and Connecting to the database

create a new folder `config` in the root folder of our application for keeping all the configurations

```
mkdir config
cd config
```

Create a new file `database.config.js` inside `config` folder with the following contents:

```
module.exports = {
  url: 'mongodb://localhost:27017/crud-node-express' }
```

import the above database configuration in `server.js` and connect to the database

```
using mongoose. Add the following code to the server.js file
after app.use(bodyParser.json()) line const dbConfig =
require('./config/database.config.js'); const mongoose = require('mongoose');
mongoose.Promise = global.Promise; mongoose.connect(dbConfig.url, {
useNewUrlParser: true }).then(() => { console.log("Database Connected Successfully!!!");
}).catch(err => { console.log('Could not connect to the database', err); process.exit(); });
```

run the server and make sure that you're able to connect to the database.

node server.js

Step 4: Create Mongoose Model

Create a folder called model inside the app folder. Create a user.js file and paste the below code.

```
var mongoose = require('mongoose'); var schema = new mongoose.Schema({ email: { type:
String, required: true, unique:
true }, firstName: { type: String, default: '' }, lastName:
{ type: String, default: '' }, phone: String, }); var user = new
mongoose.model('User', schema); module.exports = user;
```

Step 5: Create the Controller

Inside app/controllers folder, let's create User.js with these CRUD functions:

- create
- findAll
- findOne
- update
- destroy

```
const UserModel = require('../model/user') // Create and Save a new user
```

```
exports.create = async (req, res) => { if (!req.body.email && !req.body.firstName &&
!req.body.lastName && !req.body.phone) {
res.status(400).send({ message: "Content can not be empty!" }); }
```

```
const user = new UserModel({ email: req.body.email, firstName: req.body.firstName, lastName:
req.body.lastName, phone: req.body.phone });
```

```
await user.save().then(data => { res.send({ message: "User created successfully!!",
user: data }); }).catch(err => { res.status(500).send({ message: err.message || "Some error
occurred while creating user" }); }); // Retrieve all users from the database.
```

```
exports.findAll = async (req, res) => { try { const user = await UserModel.find();
res.status(200).json(user); } catch(error) { res.status(404).json({message: error.message});
} };
// Find a single User with an id
```

```
exports.findOne = async (req, res) => { try { const user = await
UserModel.findById(req.params.id); res.status(200).json(user); } catch(error) {
res.status(404).json({ message: error.message }); } }; // Update a user by the id in
the request
```

```
exports.update = async (req, res) => { if(!req.body) { res.status(400).send({ message:
"Data to update can not be empty!" }); } const id = req.params.id; await
UserModel.findByIdAndUpdate(id, req.body, { useFindAndModify: false }).then(data => { if
(!data) { res.status(404).send({ message: `User not found.` }); } else { res.send({ message:
"User updated successfully." }); } }).catch(err => { res.status(500).send({ message:
err.message }); }); } };
// Delete a user with the specified id in the request
```

```
exports.destroy = async (req, res) => { await
UserModel.findByIdAndRemove(req.params.id).then(data => { if (!data) { res.status(404).send({
message: `User not found.` }); } else { res.send({ message: "User
deleted successfully!" }); } }).catch(err => { res.status(500).send({ message: err.message });
}); });
```

Creating a new User

```
// Create and Save a new user exports.create = async (req, res) => { if (!req.body.email &&
!req.body.firstName && !req.body.lastName && !req.body.phone) { res.status(400).send({
message: "Content can not be empty!" }); } const user = new UserModel({ email:
```

```

req.body.email, firstName: req.body.firstName, lastName: req.body.lastName, phone:
req.body.phone }); await user.save().then(data => {

res.send({
    created successfully!!", message:"User user:data
    =>
});
}).catch(err {

    res.status(500).send({ message: err.message || "Some error occurred while creating user"

});
});
};

```

Retrieving all Users

```

// Retrieve all users from the database.

exports.findAll = async (req, res) => { try { const user = await UserModel.find();
res.status(200).json(user); } catch(error) { res.status(404).json({message:
error.message}); }

};

```

Retrieving a single User

```

// Retrieve all users from the database.

exports.findAll = async (req, res) => { try { const user = await UserModel.find();
res.status(200).json(user); } catch(error) { res.status(404).json({message:
error.message}); }

};

```

Updating a User

```

// Update a user by the id in the request exports.update = async (req, res) => {
if(!req.body) { res.status(400).send({ message: "Data to update can not be
empty!" }); } const id = req.params.id; await
UserModel.findByIdAndUpdate(id, req.body, { useFindAndModify: false
}).then(data { if (!data) {
=>
res.status(404).send({ message: `User not found.` }); }else{ res.send({ message: "User
updated successfully." }) } }).catch(err => { res.status(500).send({ message:
err.message }); }); };

```

Deleting a User

```
// Delete a user with the specified id in the request exports.destroy = async (req, res)
=> {
  await UserModel.findByIdAndRemove(req.params.id).then(data => {
    if (!data) { res.status(404).send({ message: 'User not found.' }); } else { res.send({
      message: "User deleted successfully!" }); } }).catch(err => { res.status(500).send({
      message: err.message }); }); });
```

Step 6: Define Routes

When a client sends a request for an endpoint using an HTTP request (GET, POST, PUT, DELETE), we need to determine how the server will respond by setting up the routes.

Create a User.js inside app/routes folder with content like this: `const express = require('express') const UserController = require('../controllers/User')`

```
const router = express.Router();
router.get('/:id', UserController.findOne);
router.post('/', UserController.create);
router.patch('/:id', UserController.update);
router.delete('/:id', UserController.destroy);
module.exports = router
```

The last step before trying out our routes is to add the route class to the `server.js`

```
const UserRoute = require('./app/routes/User')
app.use('/user', UserRoute)
```

restart your node.js server and now we have our API ready.

Conclusion: Thus, we have used node JS, express, mongo DB and mongoose libraries to create CRUD operations